PAK-AUSTRIA FACHHOCHSCHULE:
INSTITUTE OF APPLIED SCIENCES AND TECHNOLOGY

Digital Image Processing

Lab 06

Bit Plane Slicing and Histogram Processing

Submitted by:
Muhammad Hashir Sakimdad
B23F0435AI108

Submitted to:
Mr. Assad Ali (Instructor)
Mr. Khizar Hayat (Lab Engineer)

School of Computing Sciences, Fall 2026

Introduction

In this lab I worked on two basic but very useful ideas in image processing. The first one is bit plane slicing. A normal grayscale image stores every pixel in 8 bits, so each pixel value from 0 to 255 can be split into 8 separate binary images, one for each bit. Looking at these planes one by one tells us which bits actually carry the picture and which ones are mostly noise.

The second part is about histograms. A histogram simply counts how many pixels have each intensity value. From it we can tell if an image is dark, bright or washed out without even looking at it. After that I used the histogram to improve a low contrast image with two methods, min max contrast stretching and histogram equalization, and compared both.

Everything was done in Python using OpenCV, NumPy and Matplotlib. I used my own images for this lab: an outdoor group photo of me and my friends for bit plane slicing, the group photo and a code screenshot for histogram comparison, and a webcam screenshot from my ProctorAI project for the normalized histogram. For the enhancement tasks I made a hazy low contrast version of the webcam image by squeezing its pixel range to 85 to 161.

What We Learn

- How an 8 bit pixel value is built from 8 bit planes and how to pull each plane out with shift and AND operations.
- Why the higher bit planes hold the shape of the image while the lower planes look like random noise.
- How to read a histogram and decide if an image is dark or bright.
- What a normalized histogram is and why its values always add up to 1 (it is basically a probability distribution).
- How min max contrast stretching works as a simple linear mapping and where it fails.
- How histogram equalization uses the CDF to spread intensities, and when it can look unnatural.

Part A: Bit Plane Slicing

Task 1: Bit Plane Extraction

What we did: the group photo is loaded in grayscale. For bit k, the pixel value is shifted right by k positions and ANDed with 1. This leaves only that bit, so each plane is stored as 0 and 1 values. All 8 planes are then shown in a 2 x 4 grid.



Figure 1: Original grayscale image

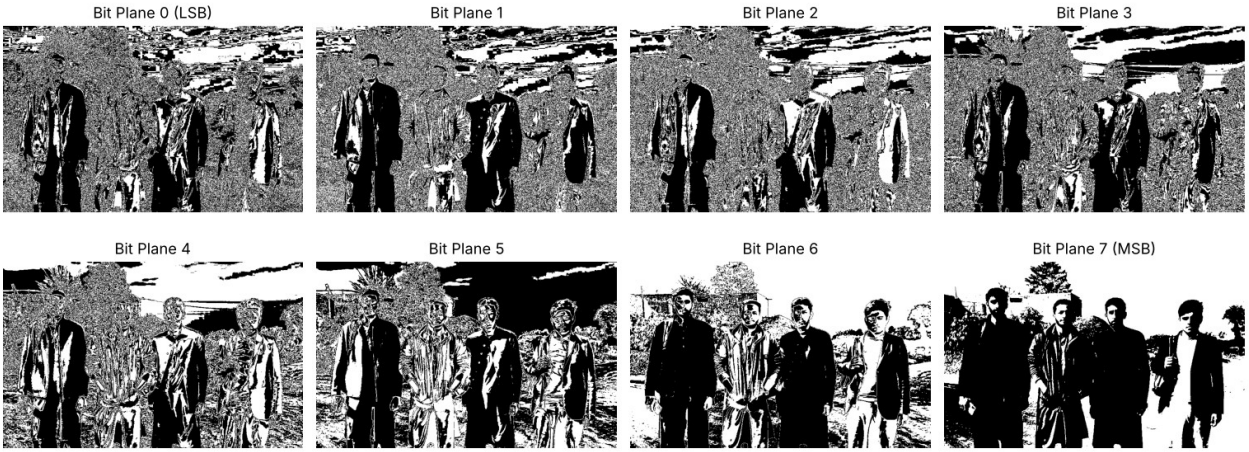


Figure 2: All eight bit planes (plane 0 is LSB, plane 7 is MSB)

Output check: every plane only contains the values [0 1], so they are stored as proper binary images.

Task 2: Comparative Analysis of Bit Planes

Planes 7, 6 and 5 hold most of the structure. In plane 7 all four people stand out clearly as dark shapes against the bright sky and the dusty road, and even the faces, the tree and the wall in the back can be recognized. Plane 6 adds more detail on the faces and clothes and the outline of the wall. Plane 5 still shows the edges of the people but the bushes and the ground start turning into texture.

Plane 4 and plane 3 are the middle area. You can still make out the four people, but the bushes and the ground become grainy and the sky breaks into blocky bands. These bands come from small intensity changes in the smooth sky.

Planes 2, 1 and 0 are mostly noise. The background in plane 0 looks like random salt and pepper. Some solid black areas still show up on the dark clothes, but that is only because those pixels are almost pure black (value 0), so all their bits are 0. It is not real detail.

Comparing high and low planes: the higher bits decide the big jumps in brightness (bit 7 alone is worth 128 levels), so they carry the shapes. The lower bits only change a pixel by 1, 2 or 4 levels, which is too small for the eye, so they mostly carry fine detail and noise. This is also why the lower planes can be removed for compression or used to hide data (steganography) without visible change.

Most recognizable single plane: bit plane 7 (MSB). It gives the clearest picture of the original on its own.

Part B: Histogram Processing

Task 3: Histogram Calculation and Interpretation

What we did: two images with different brightness were loaded and their histograms were drawn using plt.hist() with 256 bins.

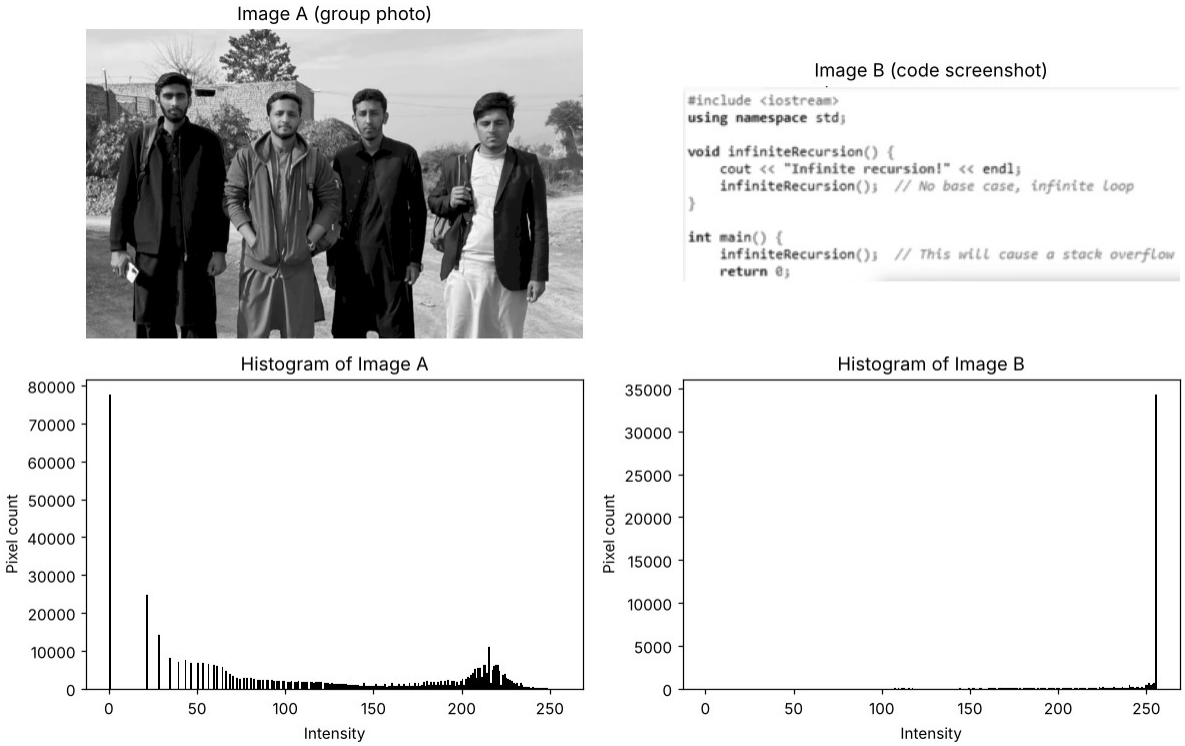


Figure 3: Two images and their intensity histograms

Result: Image A (group photo) has mean 111.76 and median 103, so it is the darker image. Its histogram has a very tall spike near 0 from the black clothes and a lot of pixels in the dark range, with a second smaller group around 200 to 230 for the sky and the road. Image B (code screenshot) has mean 244.62 and median 255, so it is clearly the brighter one. Almost all of its pixels sit in one spike at 255, which is the white background, and the text only adds a few small bars.

How the histogram shows distribution: the x axis is the intensity level from 0 (black) to 255 (white) and the y axis is the number of pixels at that level. If the bars lean left the image is dark, if they lean right it is bright, and if they are squeezed in a narrow band the image has low contrast. It does not say where the pixels are, only how many of each value exist.

Task 4: Normalized Histogram

What we did: the frequency histogram was found with cv2.calcHist, then each count was divided by the total number of pixels. This turns counts into probabilities p(r) = n(r) / (M x N). The webcam screenshot was used here.

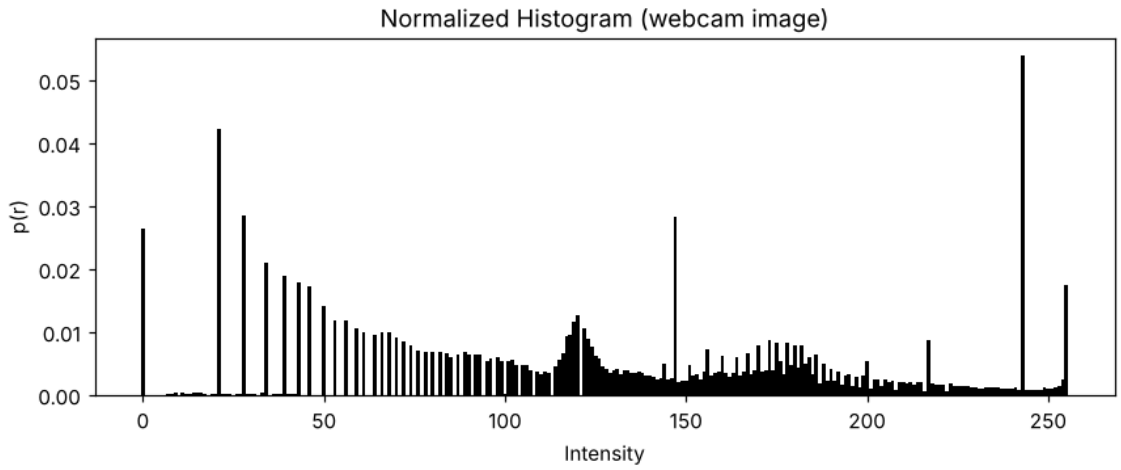


Figure 4: Normalized histogram of the webcam image

Total pixels = 319908 (503 x 636), sum of frequency histogram = 319908
Sum of normalized histogram = 1.000000
Max probability = 0.05382 at intensity 243

Verification: the sum came out exactly 1, which confirms the normalization is correct. The shape of the graph is the same as the normal histogram, only the y axis scale changed. The highest value is at intensity 243, which comes from the bright ceiling lights and the white shirt.

Task 5: Contrast Stretching

What we did: first the minimum and maximum intensity of the low contrast image were found. Then every pixel was mapped with s = (r - rmin) / (rmax - rmin) x 255 so that the darkest pixel becomes 0 and the brightest becomes 255.

Low contrast image: min = 85, max = 161, std = 20.69
Stretched image: min = 0, max = 255, std = 69.43
Distinct gray levels before = 77, after = 77

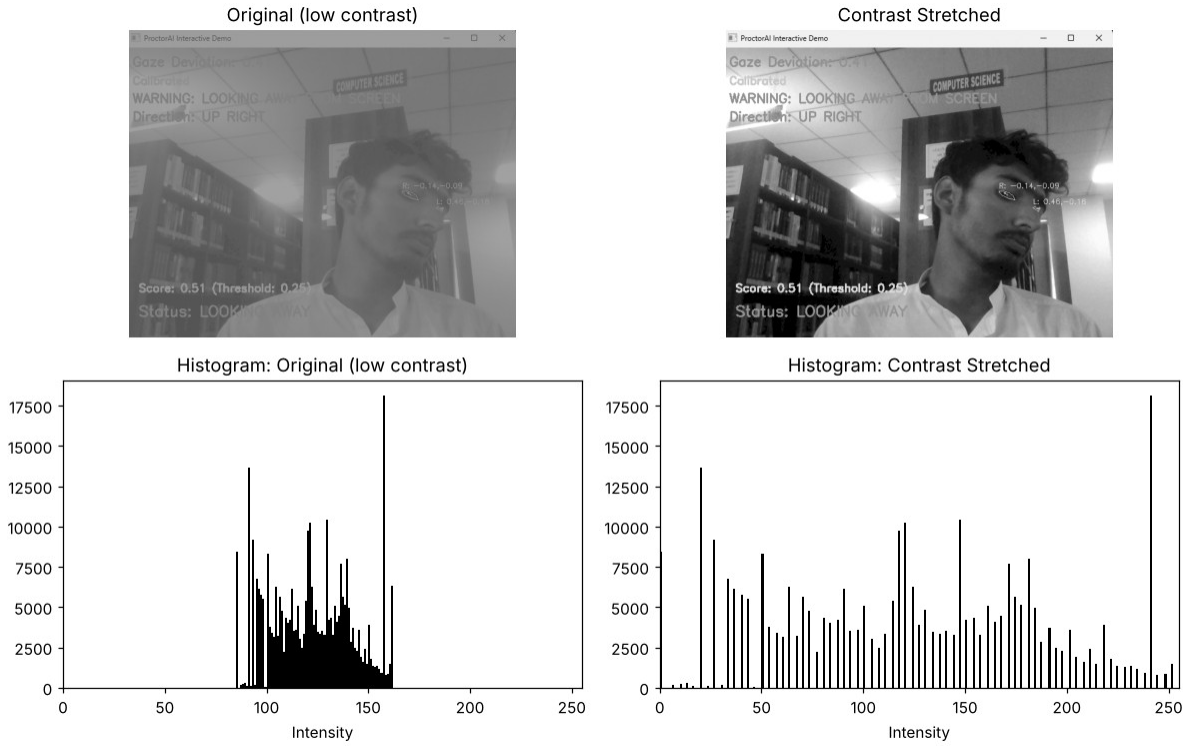


Figure 5: Original vs contrast stretched image with histograms

Comparison: the original histogram sits only between 85 and 161, which is why the picture looks flat and foggy. After stretching, the histogram covers the full 0 to 255 range and the standard deviation went from 20.69 to 69.43, so the contrast is more than three times higher. The overall shape of the histogram stayed the same, it was just pulled wider.

Limitations observed:

- The number of gray levels did not increase. There were 77 levels before and 77 after, so the histogram now has gaps (the comb look). Stretching cannot create detail that was not there.
- It depends fully on the min and max. Even one very dark or very bright noisy pixel would stop the stretch from working, because rmin or rmax would already be at 0 or 255. A percentile based stretch would handle this better.
- Since the mapping is linear, crowded regions of the histogram stay crowded relative to each other.

Task 6: Histogram Equalization

What we did: the same low contrast image was equalized using cv2.equalizeHist(). (The task sheet says the image from Task 7, but it is the same low contrast image used in Task 5.)

```
Equalized image: min = 0, max = 255, std = 75.60
Distinct gray levels after equalization = 70
```

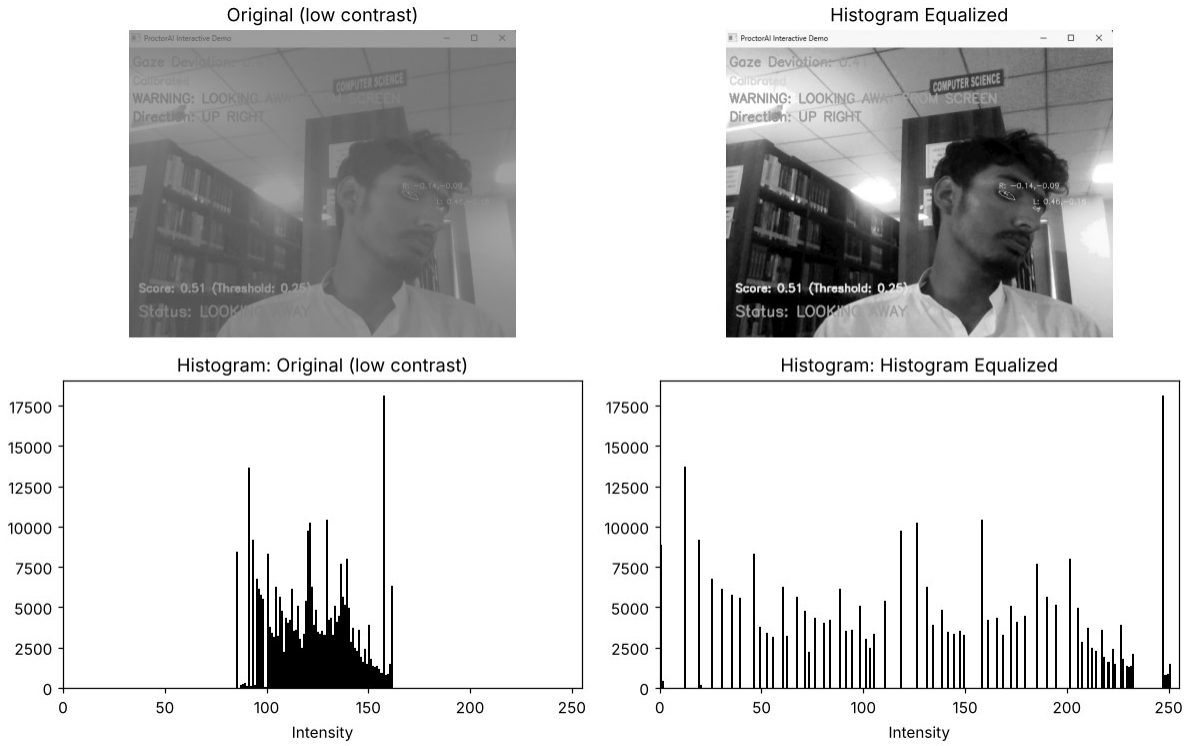


Figure 6: Original vs histogram equalized image with histograms

Comparison: visually the equalized image is much clearer than the original. The books on the shelves, the COMPUTER SCIENCE sign and the details on the face and hair all show up. In the histogram the bars are now spread across the whole range and big peaks were pushed far apart from each other, while small groups of values were merged together (70 levels left from 77).

How it improves contrast: equalization maps each level using the cumulative distribution function, s = 255 x CDF(r). Where many pixels are bunched together the CDF rises quickly, so those values get spread over a wide output range. Where few pixels exist the CDF is flat, so those levels get squeezed. The result is that the most common intensities get the most room, which is exactly where our eye needs extra contrast.

Task 7: Comparative Analysis

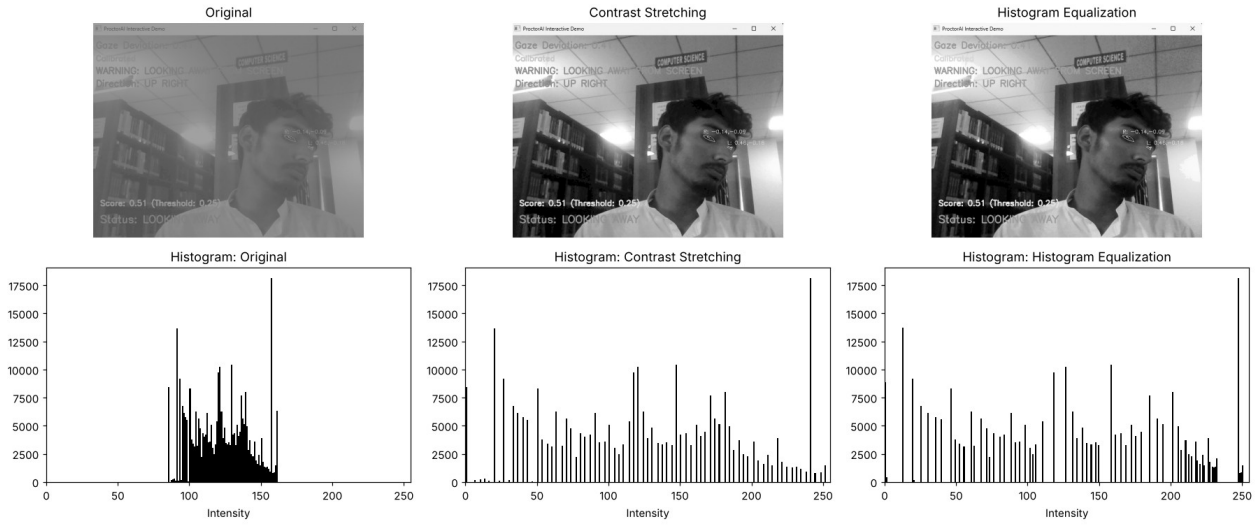


Figure 7: Original, contrast stretching and histogram equalization side by side

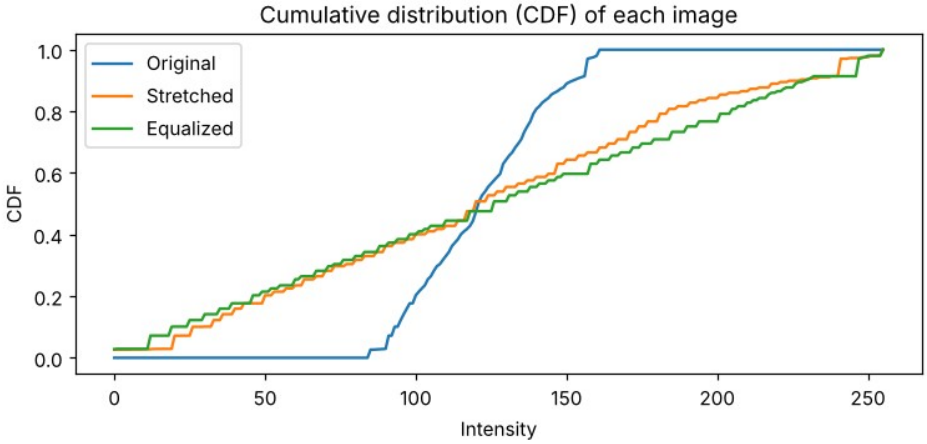


Figure 8: CDF of the three images. Equalization gives the closest to a straight line

1. Better visual enhancement: for this image histogram equalization gives a bit more punch on the bookshelves and the face, but contrast stretching looks more natural. The stretched result looks like the real webcam frame, while the equalized one has a washed out white ceiling and harsher tones. If the goal is to see hidden detail I would pick equalization, if the goal is a natural looking photo I would pick stretching.

2. Intensity distributions: stretching keeps the same histogram shape and spacing ratio, it only scales it to 0 to 255. Equalization changes the shape itself and tries to make it flat, so its CDF becomes almost a straight line (Figure 8).

3. Linear transformation: contrast stretching. It is a straight line mapping from [rmin, rmax] to [0, 255].

4. Uses the CDF: histogram equalization. The mapping function is the scaled cumulative histogram.

5. Artificial appearance: yes. In our result the ceiling and lights became almost pure white and the dark bookshelf area got grainy. Equalization boosts whatever is common, including noise in smooth areas, and it can merge levels, which gives a slightly fake or posterized look.

6. Is equalization always better? No. It works well when details are hidden in a narrow range and we just want to see them, for example in medical or satellite images. But it is not controllable, it can over enhance noise, shift the overall brightness and make photos look unnatural. Contrast stretching is predictable and keeps the relative tones, so for an image that is simply faded it is often the better and safer choice. The right method depends on the image and on what we want from it.

Conclusion

Bit plane slicing showed that most of the visible information in an 8 bit image lives in the top 3 or 4 bits, while the lower bits are close to random. In the histogram part, I learned how to read brightness from a histogram, how to normalize it into probabilities that sum to 1, and how to fix a low contrast image. Contrast stretching is simple and natural looking but limited, and histogram equalization gives stronger enhancement at the cost of sometimes looking artificial.